Modeling Pokémon Types with Deep and Lightweight CNNs: A Study in Transfer Learning and Visual Explanation

Hong Ru (Hera) Chan (hc3545)[1], Yinghao Liu (yl5645)[1], Chenyang Ren (cr3417)[1], and Yuchen Chen (yc4502)[1]

[1] Department of Statistics, Columbia University

Abstract :

This project investigates the feasibility of classifying Pokémon types using only image data, a task relevant to applications in gaming, augmented reality, and visual merchandising. Focusing on the three most frequent primary types—Grass, Water, and Normal—we curated a diverse dataset of 290 images from multiple sources to reflect visual variability across species and styles.

We implemented and compared two convolutional neural networks: EfficientNetV2B0, partially fine-tuned with augmentation and regularization; and MobileNetV2, a lightweight model with a frozen backbone. EfficientNetV2B0 outperformed MobileNetV2 in validation accuracy and generalization, aided by Grad-CAM visualizations that confirmed attention to semantically relevant regions. MobileNetV2 trained faster and required fewer resources but showed signs of overfitting and inconsistent prediction confidence.

Our findings highlight trade-offs between model complexity and deployment efficiency. We recommend EfficientNetV2B0 for high-accuracy, server-side use cases, and MobileNetV2 for low-resource environments. This work provides a foundation for future extensions including full-type coverage, multi-label classification, and real-time interactive prediction interfaces.

1 Business Problem Discussion

Visual character recognition is a growing need across industries such as gaming, augmented reality (AR), animation, education, and digital merchandising. In these domains, the ability to identify characters based solely on image input enables scalable automation, personalization, and enhanced user interaction. For example, AR applications like Pokémon Go depend on real-time object classification to create immersive experiences, while streaming platforms and retailers increasingly rely on image classifiers to auto-tag visual content, recommend products, or filter assets. In digital collectibles and gaming ecosystems, accurate character tagging can support inventory management, search functionality, and even gameplay mechanics.

As these applications scale, manual annotation becomes unsustainable, highlighting the value of robust image-based classifiers capable of generalizing across visual styles and contexts. The challenge lies in developing models that not only recognize known entities but also interpret nuanced visual features associated with abstract categories.

This project explores whether deep learning models can classify Pokémon by type using only image data—without relying on structured attributes or metadata. Unlike typical classification tasks, Pokémon types are conceptually assigned and span a wide range of appearances. For instance, Grass-type Pokémon can resemble plants, animals, or humanoid forms, while Water- and Normal-types often share similar shapes or colors. These ambiguities create a compelling testbed for evaluating whether neural networks can extract and generalize semantic patterns from diverse visual inputs—a task directly relevant to many real-world classification problems in commercial visual recognition systems.

2 Data Overview

The original dataset includes 809 Pokémon from Generations 1 through 7, with images scraped from both the Pokémon Database *Pokémon Database* n.d. and the official Pokémon website The Pokémon Company International, 2025. Each Pokémon is associated with exactly two distinct images, resulting in a total of 1618 images. These images were intentionally sourced from separate repositories to capture stylistic and contextual variety. A single image is often insufficient to represent the full range of visual depictions a Pokémon may appear in—particularly given the differences between official art, sprites, and fan-rendered styles. By merging these two sources, we constructed a dataset with improved representational richness, which is crucial for training generalizable models.

Metadata accompanying the images includes the Pokémon name, primary type (Type 1), optional secondary type (Type 2), and evolutionary form. Initially, the dataset included 18 unique type labels, including categories such as Fairy, Ghost, Dragon, and Steel. However, modeling all 18 types posed computational and data sparsity challenges, particularly given the limited number of labeled examples per class.

To streamline the classification task while maintaining relevance and challenge, we restricted the classification problem to the three most frequently occurring primary types: Grass, Water, and Normal. When a Pokémon had two types, only the primary type (Type 1) was retained for labeling purposes. This simplification enabled the use of balanced, stratified subsets while still providing meaningful variation for learning. The final filtered dataset included 290 labeled image samples, split into 232 training instances and 58 validation instances.

3 Data Exploration

Exploratory analysis of the filtered dataset, which includes only Pokémon of types **Water**, **Normal**, and **Grass**, revealed a reasonably balanced class distribution suitable for multi-class classification. As shown in **Figure 1**, the dataset contains **112 Water-type**, **104 Normal-type**, and **76 Grass-type** Pokémon. While not perfectly balanced, this distribution does not necessitate resampling or class weighting. The bar chart provides a clear overview of type proportions and supports the decision to focus on these three categories.

In addition to evaluating class balance, we conducted visual inspection of sample images within each type. This qualitative review revealed substantial *intra-class diversity* and occasional *inter-*

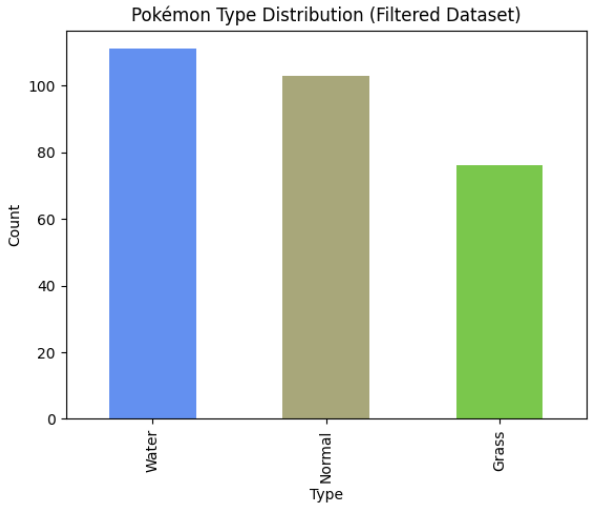

Figure 1. Pokémon type distribution in the filtered dataset.

class ambiguity. These effects were particularly pronounced for Normal-type Pokémon, which displayed a broad range of visual characteristics without strong distinguishing motifs. To illustrate this challenge, Figure 2 shows five randomly sampled images from each of the three target classes. Pokémon of the same type vary significantly in color, posture, and style, underscoring the complexity of the visual classification task.

To enhance generalization and reduce overfitting for the EfficientNetV2B0 model, we applied data augmentation to the training images using Keras' `ImageDataGenerator`. These transformations included normalization, small random rotations, shifts, zooms, shearing, and horizontal flips. Augmentation was applied exclusively to EfficientNetV2B0, as its base layers were partially fine-tuned. MobileNetV2 did not use augmentation due to its frozen convolutional backbone, which is less adaptable to altered input distributions.

4 Methodology

To rigorously assess model performance under varying design constraints, we implemented and compared two modern convolutional neural network (CNN) architectures: **EfficientNetV2B0** and **MobileNetV2**. These models were selected to represent a spectrum of trade-offs between classification accuracy and computational efficiency. **EfficientNetV2B0** prioritizes performance through fine-tuning and expressive capacity, making it suitable for server-based or offline inference. In contrast, **MobileNetV2** represents a lightweight, mobile-optimized architecture ideal for resource-constrained environments requiring real-time inference.

This dual-model framework allowed us to evaluate how transfer learning strategies—such as layer freezing, fine-tuning, regularization, and augmentation—affect model outcomes under differing resource and deployment scenarios.

4.1 EfficientNetV2B0

We initialized EfficientNetV2B0 with pretrained weights from ImageNet. To enable domain adaptation, the top 50% of the base model layers were **unfrozen** for fine-tuning. This allowed the network to adjust low- and mid-level filters to better capture Pokémon-specific visual patterns.

The classification head extended the base model with:

- A dense layer with 256 units, ReLU activation, and `L2` regularization ($\lambda = 0.01$)
- Batch normalization
- Dropout with a rate of 0.5
- A softmax output layer for 3-class prediction

Input images were resized to 256×256 pixels to align with EfficientNetV2B0's expected input size. We used the `Adam` optimizer (learning rate = 0.00001) and categorical cross-entropy loss. A rich callback strategy was adopted to stabilize training:

- **EarlyStopping**: Monitored validation loss (patience = 5), restoring best weights
- **ModelCheckpoint**: Saved model with best validation accuracy
- **ReduceLROnPlateau**: Reduced learning rate when validation loss plateaued (factor = 0.2, patience = 3)
- **TensorBoard**: Enabled real-time training metric visualization

To mitigate overfitting, we applied extensive **data augmentation** using Keras' `ImageDataGenerator`, including:

- Pixel rescaling to [0, 1]
- Random rotations (up to $\pm 20°$)
- Width and height shifts (up to 20%)
- Shear and zoom transformations (up to 20%)
- Horizontal flips

These augmentations introduced meaningful variance without altering semantic labels, allowing EfficientNetV2B0 to generalize more robustly across validation examples.

4.2 MobileNetV2

MobileNetV2 was also initialized with ImageNet weights but optimized for lightweight inference. All layers in the base model were **frozen** to prevent updates during training. This reduced training time and memory footprint, while minimizing overfitting risk given the dataset size.

A compact classification head was appended, composed of:

- A dense layer with 256 units and ReLU activation
- Batch normalization
- Dropout with a rate of 0.3
- A softmax output layer for 3-class classification

The model accepted 224×224 RGB inputs. It was compiled with the same optimizer and loss function as EfficientNetV2B0, but trained under a simplified callback setup:

- **EarlyStopping**: Based on validation loss (patience = 5)
- **ModelCheckpoint**: Saved best model by validation accuracy

No data augmentation was used during MobileNetV2 training. Since the convolutional backbone was frozen, introducing augmented transformations could create a mismatch with pre-learned filters, leading to degraded performance.

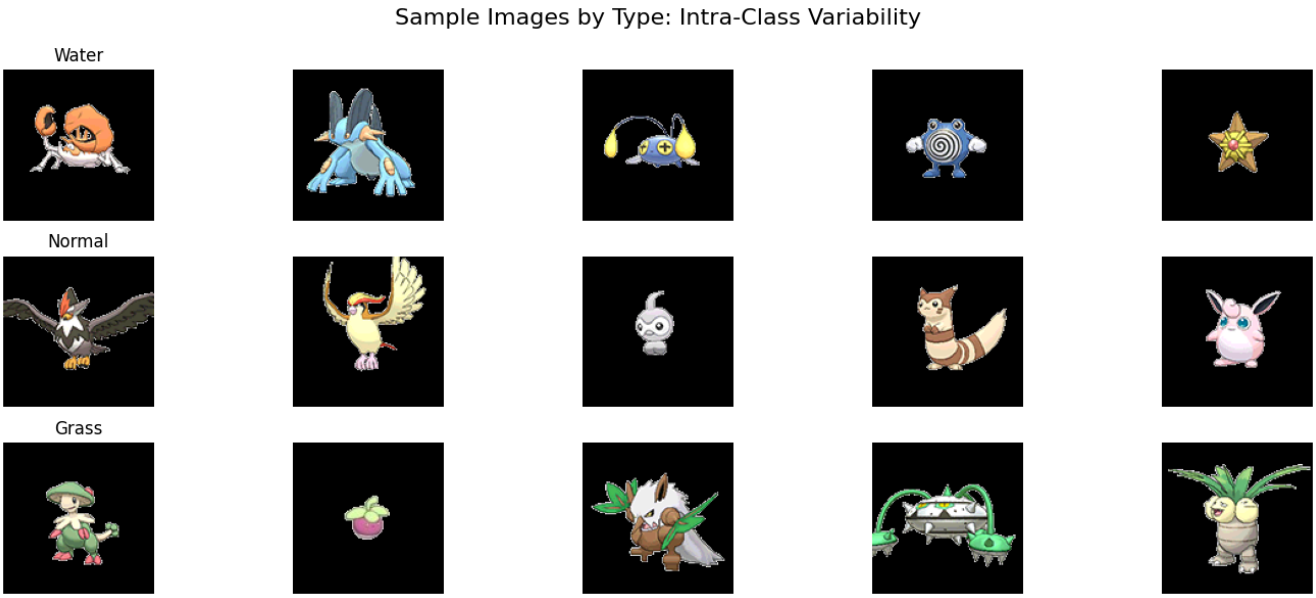

Figure 2. Sample images by type showing intra-class visual variability. Each row represents five randomly sampled Pokémon from the Water, Normal, and Grass classes respectively, highlighting the range of visual appearances within each type.

4.3 Summary

This comparative design enabled structured evaluation of two contrasting strategies:

- **EfficientNetV2B0**: A high-performance, partially fine-tuned model leveraging data augmentation and regularization to maximize generalization.
- **MobileNetV2**: A compact, efficient model with a frozen backbone optimized for quick training and low-resource inference.

Together, these models allowed us to analyze how training methodology and architectural design interact under different computational and generalization constraints in the context of Pokémon type classification.

5 Model Results and Analysis

To evaluate and compare model performance, we analyzed training dynamics, validation accuracy and loss, confusion matrices, and qualitative visualizations. Both **EfficientNetV2B0** and **MobileNetV2** demonstrated strong performance but differed significantly in generalization behavior and robustness.

5.1 Accuracy and Loss Trends: Model Comparison

We compared **EfficientNetV2B0** and **MobileNetV2** using accuracy and loss curves (Figures 3–6). Below we summarize the key observations for each model.

5.1.1 EfficientNetV2B0: Stable Learning and Generalization

- Training and validation accuracy increased consistently over epochs with a small, stable gap (Figure 3).
- Validation loss decreased smoothly alongside training loss, indicating strong convergence (Figure 4).
- The model was aided by data augmentation, dropout (0.5), L2 regularization, and training callbacks (EarlyStopping, ReduceLROnPlateau).
- These strategies led to minimal overfitting and stable generalization to unseen data.

5.1.2 MobileNetV2: Overfitting and Instability

- Training accuracy rapidly exceeded 90%, but validation accuracy plateaued and showed instability (Figure 5).
- Validation loss fluctuated across epochs and even increased in later stages, while training loss continued to drop (Figure 6).
- This indicates overfitting due to the frozen base layers, weaker regularization (Dropout 0.3), and insufficient augmentation.
- The model may benefit from:
  - Unfreezing and fine-tuning additional layers,
  - Adding stronger regularization (e.g., higher Dropout, L2 penalties),
  - More aggressive early stopping to prevent memorization.

Summary

- **EfficientNetV2B0** achieved smoother convergence and better generalization due to deeper architecture and tuning.
- **MobileNetV2** trained faster and achieved high training accuracy, but failed to maintain validation performance due to overfitting.

5.2 Class-wise Performance Comparison

Confusion matrices further illustrate each model's strengths and weaknesses. As shown in Figure 8, **EfficientNetV2B0** achieved:

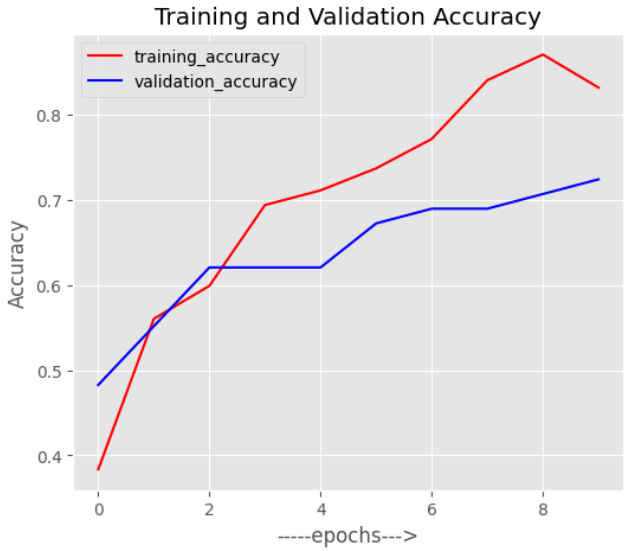

Figure 3. EfficientNetV2B0 training and validation accuracy.

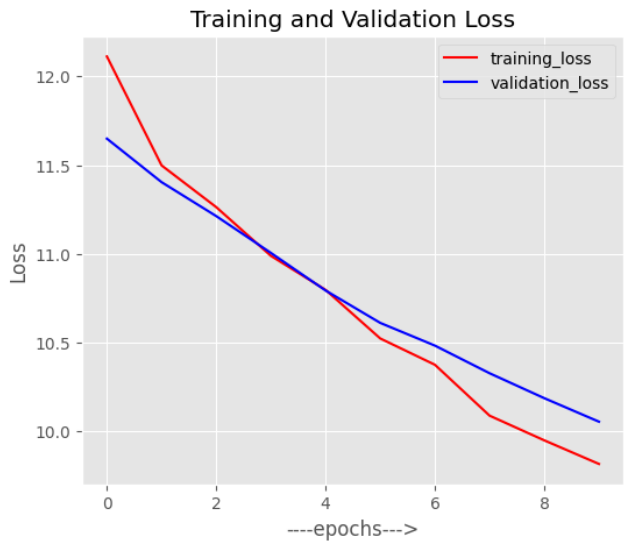

Figure 4. EfficientNetV2B0 training and validation loss.

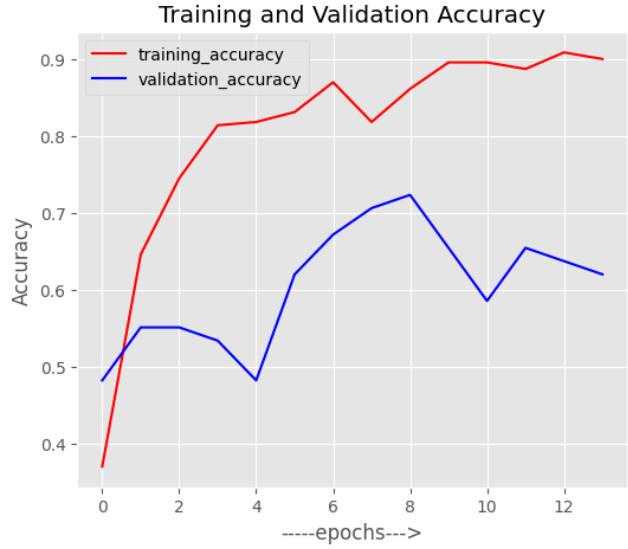

Figure 5. MobileNetV2 training and validation accuracy.

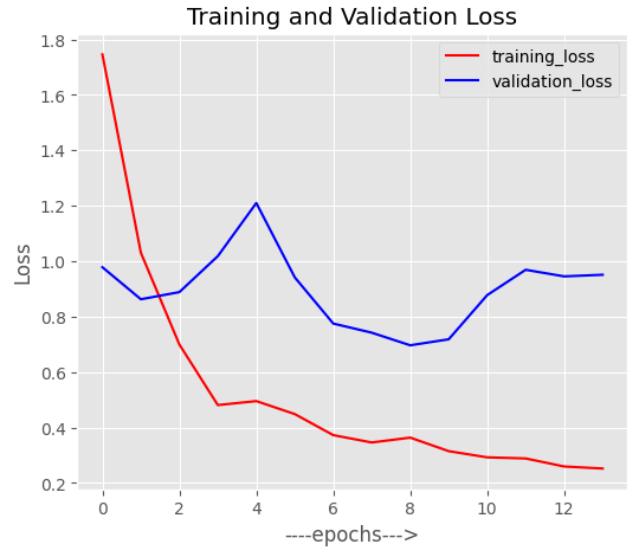

Figure 6. MobileNetV2 training and validation loss.

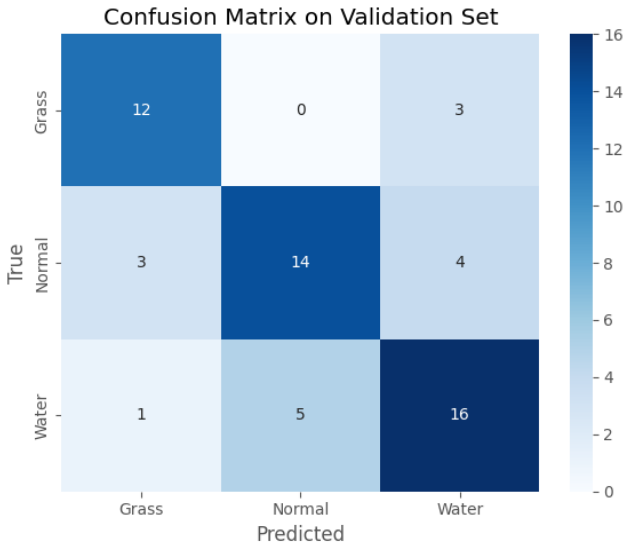

Figure 7. MobileNetV2 confusion matrix (raw counts).

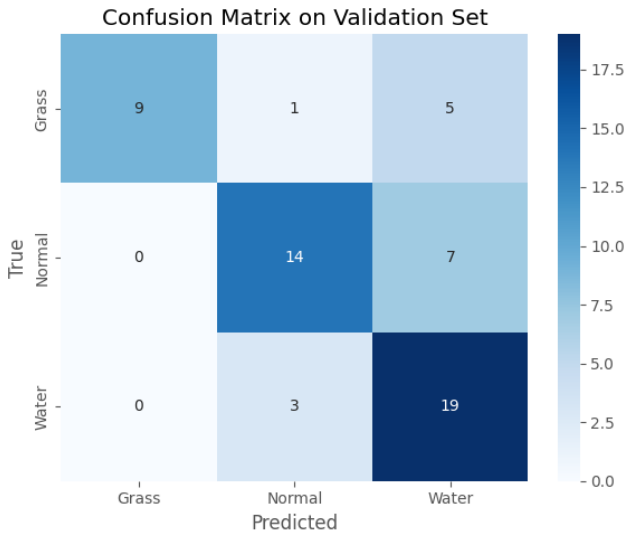

Figure 8. EfficientNetV2B0 confusion matrix (raw counts).

- 86% accuracy on **Water** types (19/22),
- 67% on **Normal** types (14/21),
- and 60% on **Grass** types (9/15).

Misclassifications for Grass were commonly predicted as Water, indicating visual overlap or ambiguity between these categories.

MobileNetV2 (Figure 7) exhibited greater confusion, particularly between Normal and Water types. While it achieved 80% accuracy on Grass (12/15), Normal and Water predictions were less consistent.

Table 1 summarizes normalized class accuracy for both models. These values were computed from raw confusion matrix counts (Figures 7 and 8).

Table 1
Normalized Validation Accuracy by Type

Pokémon Type	MobileNetV2	EfficientNetV2B0
Grass	12/15 = 80.0%	9/15 = 60.0%
Normal	14/21 = 66.7%	14/21 = 66.7%
Water	16/21 = 76.2%	19/22 = 86.4%

5.3 Qualitative Prediction Visualization

5.3.1 EfficientNetV2B0: Grad-CAM Attention Visualization

To gain interpretability into EfficientNetV2B0's predictions, we applied Grad-CAM to visualize which spatial regions influenced the model's decisions. As shown in Figure 11, the model generates a heatmap that highlights high-activation zones in the input image. In this illustrative example, Grad-CAM reveals concentrated attention on the Pokémon's head and facial features—areas typically correlated with semantic identity. This particular case demonstrates how the model leverages localized cues during inference. These visualizations help us assess whether the model is attending to meaningful regions or being distracted by background noise or irrelevant artifacts.

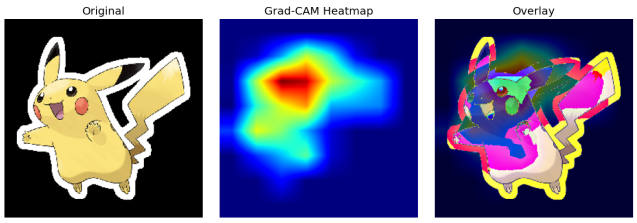

Figure 11. EfficientNetV2B0: Grad-CAM illustration of model attention. This example shows how EfficientNetV2B0 localizes semantically relevant regions in the image—such as the head and facial features, when forming a prediction. While this particular instance is not guaranteed to be correctly classified, the visualization demonstrates the model's spatial focus during inference.

Figures 9 and 10 present Grad-CAM heatmaps for misclassified examples, revealing how the model's attention led to incorrect predictions. In these failure cases, the model often focused on localized regions that were either weakly informative or visually misleading.

In Figure 9, multiple Grass-type Pokémon were incorrectly predicted as Water. Grad-CAM shows that the model concentrated on rounded, blue-colored segments—such as bulbs, petals, or shadows—that visually resemble features common to Water types. The heatmaps indicate that these blue-highlighted zones (high-activation areas) were interpreted as strong Water-type signals, likely due to color priors learned during training. However, such regions are not semantically distinctive enough to differentiate Grass from Water in these ambiguous instances.

Similarly, in Figure 10, Normal-type Pokémon were misclassified as Water, and vice versa. The model's focus was often drawn to pale or soft-colored body parts—such as pink torsos or white faces—that lack strong class identity. In some cases, the attention was diffused across the body or overly centered on non-discriminative features, which weakened the model's ability to distinguish between the two visually subtle classes.

These observations suggest that even a well-regularized, fine-

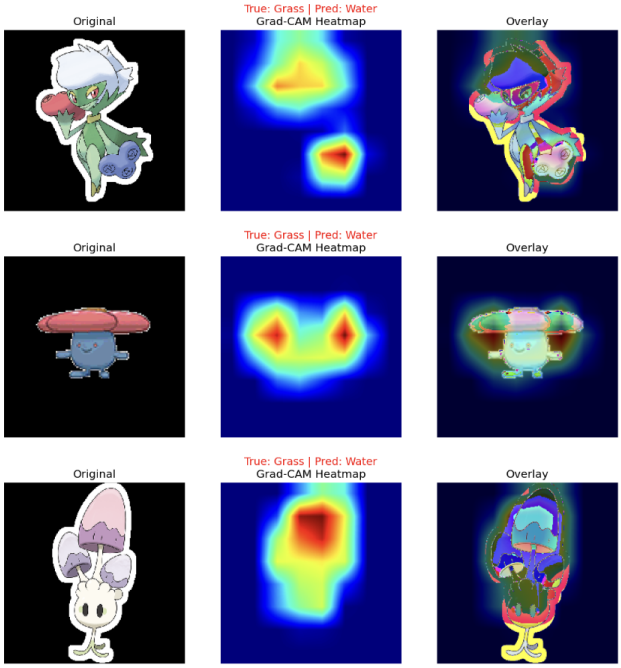

Figure 9. Misclassified Grass Pokémon Grad-CAM examples.

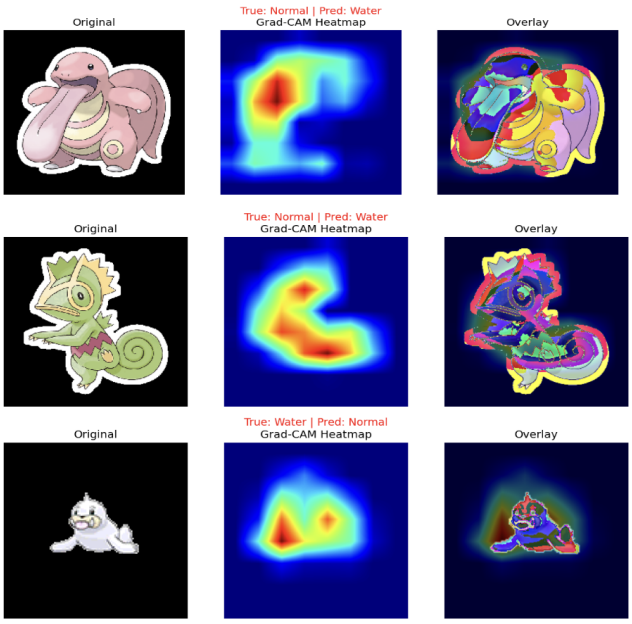

Figure 10. Misclassified Normal/Water Grad-CAM examples.

tuned model like EfficientNetV2B0 can struggle when visual attributes such as color, shape, and texture overlap across classes. The Grad-CAM visualizations help pinpoint exactly where and why the model's decision boundaries became blurred—particularly in the presence of ambiguous morphology or background artifacts.

5.3.2 MobileNetV2: Prediction Confidence Analysis Because MobileNetV2 was not fine-tuned and lacked strong regularization, we relied on raw prediction confidence scores to interpret the model's behavior. Figure 12 displays a set of representative examples showing how confident the model was in its predictions.

- **High-confidence correct predictions**: The model confidently classified Ducklett as `Water` (95.48%) and Ditto as `Normal` (95.06%), suggesting it learned to identify certain visual traits like color or silhouette that strongly associate with these types.
- **Moderate to low-confidence predictions**: In contrast, Ambipom was classified as `Normal` with only 52.52% confidence, despite being correct. This low certainty may reflect ambiguity in visual features or model hesitation due to intra-class variability.
- **General behavior**: The model correctly predicted Snivy (Grass) with 94.11% confidence, indicating that some Grass types are reliably distinguishable. However, the wide spread in prediction confidence across different examples highlights MobileNetV2's limited ability to generalize, possibly due to its frozen backbone and lack of visual attention mechanisms.

These examples underscore that while MobileNetV2 can learn type-indicative features to some extent, its confidence is inconsistent and sensitive to subtle visual differences. Without fine-tuning or interpretability tools like Grad-CAM, the model lacks robustness in distinguishing between classes with overlapping appearances.

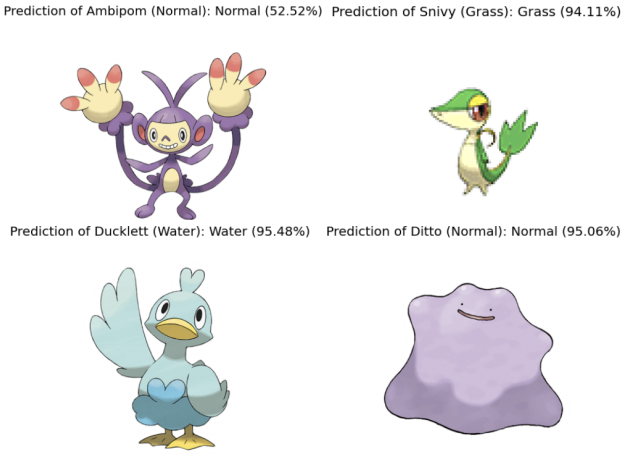

Figure 12. MobileNetV2 predictions with confidence levels.

Key Takeaways

- **EfficientNetV2B0** achieved stronger generalization and validation stability due to partial fine-tuning, regularization (dropout, L2), and extensive data augmentation.
- **MobileNetV2** trained faster with fewer resources, but its frozen backbone and lack of augmentation led to overfitting and inconsistent validation performance.
- Model interpretability techniques—Grad-CAM for EfficientNetV2B0 and confidence scoring for MobileNetV2—provided complementary insights into decision-making patterns and

failure cases.

- Both models classified `Water`-type Pokémon with high accuracy, while `Grass` and `Normal` types were more frequently confused, often due to overlapping visual features or ambiguous morphology.
- Visual attention maps revealed that misclassifications often arose from focus on color-dominant or shape-similar regions (e.g., blue petals being interpreted as Water-type cues).

6 Conclusion & Recommendations

This project evaluated the feasibility of using deep learning to classify Pokémon types based solely on image data, targeting the three most common types: Grass, Water, and Normal. By comparing two modern CNN architectures—EfficientNetV2B0 and MobileNetV2—we quantified trade-offs between model complexity, performance, and deployment efficiency.

EfficientNetV2B0 achieved the highest validation accuracy and demonstrated better generalization to visually similar examples, particularly through fine-tuning and strong regularization. Its attention maps (via Grad-CAM) revealed a focus on semantically meaningful regions, supporting its interpretability. These characteristics make it ideal for high-accuracy use cases such as:

- Server-based batch classification of new Pokémon cards.
- Internal QA tools for validating label accuracy in a growing image database.

MobileNetV2, while slightly less accurate, offered substantial benefits in training speed and model size. Its efficiency and simplicity make it a practical choice for:

- Mobile apps for casual users identifying Pokémon from camera input.
- Low-resource deployments or embedded environments, such as browser-based or on-device inference.

Both models accurately identified Water-type Pokémon but showed confusion across Grass and Normal types—often due to shared color patterns or ambiguous features. These insights suggest that additional data diversity and model interpretability tools will be essential for handling a full 18-class taxonomy or dual-type classification tasks.

Recommendation: Deploy EfficientNetV2B0 for centralized applications where accuracy and interpretability are essential. Use MobileNetV2 for lightweight, client-facing use cases where responsiveness and model size are more critical. These models can serve as the foundation for a scalable image-based classification system with future extensions to broader Pokémon types, real-time prediction interfaces, and multi-label outputs.

7 Limitations and Future Work

This project served as a focused proof-of-concept for visual Pokémon type classification using deep learning. To scale this work into a robust, real-world system, several extensions are recommended:

1. **Expand to all 18 Pokémon types**: Transitioning from three to all canonical types would create a more realistic and challenging multi-class classification setting.
2. **Support multi-label classification**: Many Pokémon belong to dual types (e.g., Water/Flying), necessitating architectures that can handle multi-label outputs and overlapping semantic classes.
3. **Broaden data sources**: Incorporating fan art, in-game sprites, and 3D renderings would improve robustness across diverse visual styles and better simulate real-world use cases.
4. **Deploy a web or mobile interface**: Building an interactive frontend would enable user-facing applications, such as real-time classification from camera input or image upload.
5. **Explore vision transformers (ViTs)** and **self-supervised learning**: These architectures have demonstrated superior performance on small or unlabeled datasets and may improve generalization when labeled Pokémon data is limited.
6. **Use cross-validation or bootstrap resampling**: These techniques would yield more reliable performance estimates and help assess model variance, especially in data-limited settings.
7. **Introduce model ensembles**: Combining predictions from CNNs and transformer-based models may increase classification stability and reduce type confusion.

Together, these directions would transform the current prototype into a scalable, general-purpose visual recognition system—capable of handling diverse inputs, complex label structures, and real-time user interaction.

Acknowledgements

This project received support during the STAT GR 5243: Applied Data Science course, instructed by Dr. Haiyuan Wang at Columbia University.

References

Pokémon Database (n.d.). https://pokemondb.net/pokedex/national. Accessed April 2025.

The Pokémon Company International (2025). *Pokémon Pokédex*. https://www.pokemon.com/us/pokedex. Accessed April 2025.

Appendix

Code available at: github.com/hera-chan/pokemon-cnn-classification-gradcam/